

Supplementary Material

Making Critical DNA Evidence Accessible: What Current Statistics Can Leave Unsaid for the Defense

With an AI Assistant and an Invitation to Open Review

This supplement accompanies the manuscript bearing the title above. It contains the complete calculation script, instructions for running it, the numerical-results record, and a map of the supporting material.

Use your supplied copy of the current `manuscript.md` or its corresponding PDF. The current `manuscript.md` is the controlling text; the PDF is a static distribution copy. The manuscript contains the mathematical definitions, assumptions, derivations, and interpretive boundaries.

Material	Repository location
This supplement, <code>supplemental.md</code>	Supplemental materials repository
Repository guide and article-focused AI instructions, <code>rob.v1.md</code>	Rob repository
Separate vector figures, <code>figure-1.svg</code> through <code>figure-4.svg</code>	Figures repository

These links identify repositories, not direct file downloads or a public manuscript download location. Open the relevant repository and obtain the named file. A repository link alone does not establish that an AI system has accessed its contents.

The current manuscript Markdown embeds the figures, and its corresponding PDF includes them. Separate SVG files are available for full-size viewing and access to the original vector figures; they are not required merely to display the embedded figures in a compatible viewer.

All examples are synthetic. The script requires no external dataset or third-party Python packages.

S1. Map of supporting material

Material	Manuscript location	Implementation or supporting file
Strict two-contributor construction and selected separate and joint LR _s	Sections 3 and 14.1	<code>strict()</code> in Section S3; Figure 1
Genotype posteriors and matched posterior–LR identities	Section 4	<code>strict()</code> and <code>quantitative()</code> in Section S3
Population genotype filters	Section 5	<code>strict()</code> in Section S3; Figure 2
Example D: fixed rarity with changing minor signal	Sections 6 and 14.2–14.3	<code>quantitative()</code> in Section S3; Figure 3

Material	Manuscript location	Implementation or supporting file
Example G: changing quantitative patterns with unknown co-contributors	Sections 7 and 14.2–14.3	<code>quantitative()</code> in Section S3; Figure 4
Posterior recovery, uniform-reference comparisons, and multilocus alternatives	Appendix A	Selected finite examples and formula checks in <code>supporting()</code> in Section S3
Occupancy, source attribution, nuisance treatment, population filters, and conditional relatedness	Appendix B	<code>supporting()</code> in Section S3
Numerical results and programmed-check totals	Section 14.4	Section S4
Figure files and their interpretation	Sections 3.4, 5, 6, and 7	Section S5

The manuscript contains the mathematical definitions, assumptions, derivations, and interpretive boundaries. This supplement provides the implementation and numerical record; it does not replace those specifications.

S2. Running the calculations

S2.1 Requirements and execution

Use Python 3. The recorded results in Section S4 were obtained using CPython 3.11.14.

Copy the complete Python block in Section S3 into a plain-text file named `reproduce.py`, preserving indentation. Save it using UTF-8 encoding.

Run:

```
python3 reproduce.py
```

On systems where the Python 3 command is `python`, use:

```
python reproduce.py
```

The script prints the Python version and implementation, numerical results, section-level check totals, and an overall total. It exits with status zero if all checks pass and status one if any programmed check fails. Failed checks are identified by name.

To save the output:

```
python3 reproduce.py > execution.txt
```

The files `reproduce.py` and `execution.txt` are local working files created by these instructions. Their contents are included in this supplement as code and a numerical-results record; separate downloads are not required.

S2.2 Numerical implementation

The strict construction uses exact rational arithmetic through Python’s `Fraction` class. Unknown-person genotypes follow the specified Hardy–Weinberg distribution. All ordered contributor pairs are enumerated.

Examples D and G use exact rational inputs before conversion to floating-point likelihood calculations. Gaussian likelihoods are evaluated on the log scale, and marginalization uses log-sum-exp. The omitted Gaussian normalization factor is common to the configurations within each setting.

For D, the major genotype is fixed and all twenty-one target-position genotypes are evaluated. For G, all twenty-one target-position genotypes and all ordered pairs of co-contributor genotypes are evaluated. Co-contributor weights are population probabilities.

The script computes the different-genotype comparison B directly from the alternative likelihood sum rather than by subtracting a near-one posterior from one. It separately checks agreement with the posterior transformation.

Printed floating-point values can differ in their final digits across environments. The script includes explicit tolerances for numerical identities and comparisons with displayed values.

S2.3 Scope of the checks

The 132 programmed checks comprise:

Group	Checks	Scope
Strict construction and population filters	22	Exact enumeration, probabilities, assignment symmetry, profile calculations, and matched identities
Six D/G settings	78	Inputs, generated signals, posterior normalization, identities, bounds, finite log likelihoods, and displayed numerical values
Supporting examples	32	Selected finite enumerations and direct formula evaluations from Appendices A and B
Total	132	Related assertions, not independent validations

Supporting checks include direct evaluations of the source-posterior, occupancy, multilocus population-filter, and conditional-relatedness formulas. Such checks confirm the stated arithmetic, not the suitability of the assumptions for casework.

The script covers the specified finite examples. It does not certify every general derivation, validate laboratory methods or commercial software, measure reader comprehension, or establish legal conclusions.

S3. Complete calculation script

```
#!/usr/bin/env python3
"""Numerical examples for:
Making Critical DNA Evidence Accessible:
What Current Statistics Can Leave Unsaid for the Defense
With an AI Assistant and an Invitation to Open Review

Python standard library only. Run: python3 reproduce.py
Strict calculations use Fraction; Gaussian calculations use float/log-sum-exp.
Tests cover specified finite examples, not all derivations or applications.
"""

import math
import platform
from fractions import Fraction as F
from itertools import combinations_with_replacement, product

CHECKS = []

def check(name, condition):
    CHECKS.append((name, bool(condition)))

def near(name, actual, expected, rel=1e-10, absolute=1e-12):
    check(name, math.isclose(float(actual), float(expected),
                             rel_tol=rel, abs_tol=absolute))

def rounded(name, actual, expected, half_unit):
    # A small floating-point allowance, not a model-error tolerance.
    check(name, abs(float(actual) - expected) <= half_unit + 1e-12)

def show(name, value):
    text = format(value, '.15g') if isinstance(value, float) else str(value)
    print(f'{name}: {text}')

def states(freq):
    gs = list(combinations_with_replacement(range(len(freq)), 2))
    ps = [freq[a] * freq[b] * (1 if a == b else 2) for a, b in gs]
    return gs, ps

def lse(values):
    values = list(values)
    m = max(values)
    return m + math.log(math.fsum(math.exp(v - m) for v in values))
```

```

def strict():
    print('\nSTRICT: exact enumeration')
    freq = list(map(F, ['0.07', '0.08', '0.09', '0.10', '0.66']))
    gs, ps = states(freq)
    obs, a, b = set(range(4)), (1, 2), (0, 2) # Zero-based alleles.
    fit = lambda g, h: set(g) | set(h) == obs
    pairs = [(i, j) for i, g in enumerate(gs)
              for j, h in enumerate(gs) if fit(g, h)]
    den = sum((ps[i] * ps[j] for i, j in pairs), F(0))
    ls = [sum((p for h, p in zip(gs, ps) if fit(g, h)), F(0))
          for g in gs]
    post = [p * ll / den for p, ll in zip(ps, ls)]
    la, lb = ls[gs.index(a)], ls[gs.index(b)]
    ra, rb = la / den, lb / den
    check('strict population normalized', sum(ps) == 1)
    check('strict 15 genotypes', len(gs) == 15)
    check('strict 225 ordered pairs', len(gs) ** 2 == 225)
    check('strict six compatible ordered pairs', len(pairs) == 6)
    check('strict equal compatible prior products',
          len({ps[i] * ps[j] for i, j in pairs}) == 1)
    check('strict denominator', den == F('0.0012096'))
    check('strict A numerator', la == F('0.014'))
    check('strict B numerator', lb == F('0.016'))
    check('strict joint likelihood zero', not fit(a, b))
    check('strict six equal positive position posteriors',
          sorted(p for p in post if p) == [F(1, 6)] * 6)
    check('strict posterior normalized', sum(post) == 1)
    check('strict matched identities all genotypes',
          all(ll / den == w / p for ll, w, p in zip(ls, post, ps)))
    check('strict A assignment symmetry',
          la == sum(p for g, p in zip(gs, ps) if fit(g, a)))
    check('strict B assignment symmetry',
          lb == sum(p for g, p in zip(gs, ps) if fit(g, b)))
    check('strict profile count', 6 ** 13 == 13060694016)
    near('strict A full profile', ra ** 13, 6.688386286845605e13)
    near('strict B full profile', rb ** 13, 3.795042627926862e14)
    near('strict profile posterior', F(1, 6) ** 13, 7.656560966629723e-11,
         absolute=1e-24)
    check('strict multilocus matched identity',
          ra ** 13 == (F(1, 6) / ps[gs.index(a)]) ** 13)
    comp = sum(p for g, p in zip(gs, ps) if any(fit(g, h) for h in gs))
    inside = sum(p for g, p in zip(gs, ps) if set(g) <= obs)
    check('strict target population mass', ps[gs.index(a)] == F('0.0144'))
    check('strict compatibility mass', comp == F('0.0862'))
    check('strict inside-set mass', inside == F('0.1156'))
    for name, val in [('denominator', den), ('A numerator', la),
                     ('B numerator', lb), ('A one-locus LR', float(ra)),
                     ('B one-locus LR', float(rb)), ('joint LR', 0),
                     ('A thirteen-locus LR', float(ra ** 13)),
                     ('B thirteen-locus LR', float(rb ** 13)),
                     ('position profile posterior', float(F(1, 6) ** 13)),
                     ('compatible position profiles', 6 ** 13),
                     ('compatibility mass', float(comp)),
                     ('inside-set mass', float(inside))]:

```

```
show(name, val)
```

```
def quantitative():
    print('\nD/G: full genotype enumeration with log-sum-exp')
    freq = list(map(F, ['0.30', '0.24', '0.18', '0.13', '0.09', '0.06']))
    gs, exact_ps = states(freq)
    lp = [math.log(float(p)) for p in exact_ps]
    dose = [[F(g.count(k), 2) for k in range(6)] for g in gs]
    major, target = gs.index((0, 1)), gs.index((4, 5))
    generating = [target, gs.index((2, 3)), major]
    ft, sigma = exact_ps[target], float(F('0.035'))
    cases = [
        ('D1', (F(1), F(0)), (F(1,2), F(1,2), F(0), F(0), F(0), F(0))),
        ('D6', (F(8,9), F(1,9)), (F(4,9), F(4,9), F(0), F(0), F(1,18), F(1,18))),
        ('D10', (F(4,5), F(1,5)), (F(2,5), F(2,5), F(0), F(0), F(1,10), F(1,10))),
        ('G1', (F(1,3),)*3, (F(1,6),)*6),
        ('G4', (F(19,45), F(29,90), F(23,90)),
            (F(23,180), F(23,180), F(29,180), F(29,180), F(19,90), F(19,90))),
        ('G10', (F(3,5), F(3,10), F(1,10)),
            (F(1,20), F(1,20), F(3,20), F(3,20), F(3,10), F(3,10)))
    ]
    # Main-table values and half-units of their last displayed decimal places.
    expected = {
        'D1': (1., 0.047619, 0., 0.0000005),
        'D6': (27.9507, 0.541729, 0.00005, 0.0000005),
        'D10': (91.9453, 0.997158, 0.00005, 0.0000005),
        'G1': (6.17284, 0.201837, 0.000005, 0.0000005),
        'G4': (63.6999, 0.826856, 0.00005, 0.0000005),
        'G10': (92.5924, 0.99999875, 0.00005, 0.000000005)
    }
    detailed = {
        'D1': (1., 0.0476190476190),
        'D6': (27.9507017179652, 0.5417285249059),
        'D10': (91.9452838579187, 0.9971583061431),
        'G1': (6.17283949624454, 0.2018372361458),
        'G4': (63.6999391814617, 0.8268558198124),
        'G10': (92.5923708630398, 0.9999987479771)
    }
    check('D/G 21 genotypes and normalized population',
        len(gs) == 21 and sum(exact_ps) == 1)
    check('D/G target frequency', ft == F('0.0108'))
    for name, phi, y in cases:
        ids = [major, target] if name.startswith('D') else generating
        check(name + ' fraction sum', sum(phi) == 1)
        check(name + ' exact generated signal',
            tuple(sum(phi[j] * dose[i][k] for j, i in enumerate(ids))
                for k in range(6)) == y)
        # Evaluate floats after exact input and generator checks.
        contributions = [[[float(f * x) for x in d] for d in dose] for f in phi]
        yf = list(map(float, y))

    def kernel(indices):
        mu = [math.fsum(contributions[j][i][k] for j, i in enumerate(indices))
```

```

        for k in range(6)]
    return -math.fsum((v - m) ** 2 for v, m in zip(yf, mu)) / (2 * sigma
** 2)

    if name.startswith('D'):
        ll = [kernel((major, i)) for i in range(21)]
    else:
        ll = [lse(lp[j] + lp[k] + kernel((i, j, k))
                for j, k in product(range(21), repeat=2)) for i in
range(21)]
    zu, zp = lse(ll), lse(p + v for p, v in zip(lp, ll))
    wu = [math.exp(v - zu) for v in ll]
    wp = [math.exp(p + v - zp) for p, v in zip(lp, ll)]
    r = math.exp(ll[target] - zp)
    # Direct alternative sum avoids subtracting a near-one posterior.
    b = math.exp(math.log(20) + ll[target] - lse(v for i, v in enumerate(ll)
if i != target))
    near(name + ' uniform posterior sum', math.fsum(wu), 1)
    near(name + ' population posterior sum', math.fsum(wp), 1)
    near(name + ' matched identity', r, wp[target] / float(ft))
    near(name + ' uniform comparison identity', b, 20 * wu[target] / (1 -
wu[target]), rel=1e-8)
    check(name + ' frequency bound', r <= 1 / float(ft) + 1e-10)
    check(name + ' all log likelihoods finite', all(math.isfinite(x) for x in
ll))
    er, ew, tr, tw = expected[name]
    rounded(name + ' displayed R', r, er, tr)
    rounded(name + ' displayed uniform weight', wu[target], ew, tw)
    near(name + ' detailed R', r, detailed[name][0])
    near(name + ' detailed uniform weight', wu[target], detailed[name][1])
    if name == 'D1':
        check('D1 equal log likelihoods', len(set(ll)) == 1)
        near('D1 population posterior equals prior', wp[target], ft)
    if name == 'D6':
        rounded('D6 population posterior display', wp[target], 0.301868,
0.0000005)
    if name == 'G10':
        near('G10 detailed B', b, 15974128.318833357, rel=1e-8)
    print(f'{name}: R={r:.15g}; w_U={wu[target]:.15g}; w_pi={wp[target]:.15g};
B={b:.15g}')

def supporting():
    print('\nSUPPORTING EXAMPLES: enumeration or explicit formulas as indicated')
    # A.1: enumerate the specified seven allowed configurations.
    allowed = [(i, j) for i, j in product(range(3), repeat=2)
                if (i, j) not in [(0, 0), (2, 2)]]
    marginal = [F(sum(i == k for i, j in allowed), 7) for k in range(3)]
    check('A1 marginal', marginal == [F(2, 7), F(3, 7), F(2, 7)])
    # A.2: two finite-state counterexamples.
    prior = list(map(F, ['0.1', '0.8', '0.1']))
    for label, ls, expected in [('original', ['0.6', '0.3', '0.1'], 1.935484),
                                ('swapped', ['0.6', '0.1', '0.3'], 3.529412)]:
        ls = list(map(F, ls))

```

```

    b, r = 2 * ls[0] / sum(ls[1:]), ls[0] / sum(p*l for p,l in zip(prior, ls))
    check('A2 ' + label + ' B', b == 3)
    rounded('A2 ' + label + ' R', r, expected, 0.0000005)
    show('A2 ' + label + ' R', float(r))
    rounded('A3 universe multiplier', 324 ** 20, 1.6251753e50, 0.00000005e50)
    check('A4 rounded posterior bound', 54 * F('0.99995') / F('0.00005') ==
1079946)
    # A.5: enumerate all observation probabilities and both alternatives.
    n, hit = 55, F('0.84')
    miss = (1 - hit) / (n - 1)
    numer = [hit] + [miss] * (n - 1)
    alt_gen = [miss] + [(hit + (n - 2) * miss) / (n - 1)] * (n - 1)
    alt_person = [F(1, n)] * n
    bs = [p/q for p,q in zip(numer, alt_gen)]
    rs = [p/q for p,q in zip(numer, alt_person)]
    check('A5 distributions normalized', sum(numer) == sum(alt_gen) ==
sum(alt_person) == 1)
    check('A5 B expectation under genotype alternative', sum(b*q for b,q in
zip(bs, alt_gen)) == 1)
    check('A5 R expectation under person alternative', sum(r*q for r,q in zip(rs,
alt_person)) == 1)
    eb = sum(b*q for b,q in zip(bs, alt_person))
    for name, val, exp, tol in [
        ('A5 target B', bs[0], 283.5, 0),
        ('A5 other B', bs[1], 0.160475483, 0.0000000005),
        ('A5 target event genotype alternative', alt_gen[0], 0.002962963,
0.0000000005),
        ('A5 B person-alternative mean', eb, 5.312103, 0.0000005),
        ('A5 target R', rs[0], 46.2, 0),
        ('A5 other R', rs[1], 0.162962963, 0.0000000005)]:
        rounded(name, val, exp, tol)
        show(name, float(val))
    # A.6: four-state factored model with likelihoods proportional to 4 and 1.
    likelihoods = [F(x*y) for x,y in product((4,1), repeat=2)]
    check('A6 target posterior', likelihoods[0] / sum(likelihoods) == F('0.64'))
    check('A6 every-locus alternative', likelihoods[0] / likelihoods[-1] == 16)
    rounded('A6 whole-profile alternative', 3*likelihoods[0]/sum(likelihoods[1:]),
5.333333, 0.0000005)
    # B.1: explicit source-posterior and occupancy formulas, not population
enumeration.
    f = F(1, 1000000)
    for k, exp, tol in [(100, 0.999901, 0.0000005),
                        (1000, 0.990100, 0.0000005),
                        (100000, 0.50000025, 0.000000005)]:
        val = 1 / (1 + (k-1)*f)
        rounded(f'B1 source posterior K={k}', val, exp, tol)
        show(f'B1 source posterior K={k}', float(val))
    occupancy = -math.expm1(999999 * math.log1p(-float(f)))
    rounded('B1 occupancy', occupancy, 0.632120, 0.0000005)
    show('B1 occupancy', occupancy)
    # B.2: enumerate 55 genotypes; arbitrary positive split of unseen mass.
    freq = list(map(F, ['0.07', '0.08', '0.09', '0.10'])) + [F('0.11')] * 6
    gs, ps = states(freq)
    fit = lambda g,h: set(g) | set(h) == set(range(4))

```

```

ll = [sum((p for h,p in zip(gs,ps) if fit(g,h)), F(0)) for g in gs]
target = gs.index((1,2))
uniform_ll = [F(sum(fit(g,h) for h in gs), len(gs)) for g in gs]
bw = 54 * ll[target] / (sum(ll) - ll[target])
bu = 54 * uniform_ll[target] / (sum(uniform_ll) - uniform_ll[target])
check('B2 genotype count and population sum', len(gs) == 55 and sum(ps) == 1)
check('B2 six complementary likelihoods', sorted(x for x in ll if x) ==
      sorted(map(F, ['0.0180', '0.0160', '0.0144', '0.0140', '0.0126', '0.0112'])))
rounded('B2 population nuisance B', bw, 10.470914, 0.0000005)
check('B2 uniform nuisance B', bu == F('10.8'))
show('B2 population nuisance B', float(bw))
show('B2 uniform nuisance B', float(bu))
# B.3 and B.4: direct formula checks, not new generative-model validation.
for base, exp, tol in [('0.0144', 2.373763e-28, 0.0000005e-28),
                      ('0.0862', 1.077976e-16, 0.0000005e-16),
                      ('0.1156', 8.797667e-15, 0.0000005e-15)]:
    val = float(F(base) ** 15)
    check('B3 ' + base, abs(val-exp) <= tol)
    show('B3 ' + base + ' power 15', val)
child = F('0.28') * F('0.29') * F('0.32')
check('B4 child', child == F('0.025984'))
check('B4 unrelated', child ** 2 == F('0.000675168256'))
show('B4 child', float(child))
show('B4 unrelated', float(child ** 2))

def main():
    print('Numerical examples for:\n'
          'Making Critical DNA Evidence Accessible: '\n
          'What Current Statistics Can Leave Unsaid for the Defense\n'
          'With an AI Assistant and an Invitation to Open Review')
    show('Python', platform.python_version())
    show('Implementation', platform.python_implementation())
    for task in (strict, quantitative, supporting):
        before = len(CHECKS)
        task()
        section = CHECKS[before:]
        print(f'Checks in section: {sum(ok for _,ok in section)}/{len(section)}
passed')
    failures = [name for name, ok in CHECKS if not ok]
    print(f'\nTOTAL: {len(CHECKS)-len(failures)}/{len(CHECKS)} checks passed')
    for name in failures:
        print('FAIL:', name)
    print('Check count includes related assertions, not independent validations.')
    raise SystemExit(1 if failures else 0)

if __name__ == '__main__':
    main()

```

S4. Numerical-results record

The following reproduces the numerical portion of the recorded execution output. The execution environment was CPython 3.11.14. This is a numerical-results record, not a byte-for-byte transcript of the script's complete output header.

```

STRICT: exact enumeration
denominator: 189/156250
A numerator: 7/500
B numerator: 2/125
A one-locus LR: 11.5740740740741
B one-locus LR: 13.2275132275132
joint LR: 0
A thirteen-locus LR: 66883862868456
B thirteen-locus LR: 379504262792686
position profile posterior: 7.65656096662972e-11
compatible position profiles: 13060694016
compatibility mass: 0.0862
inside-set mass: 0.1156
Checks in section: 22/22 passed

D/G: full genotype enumeration with log-sum-exp
D1: R=1; w_U=0.0476190476190476; w_pi=0.0108; B=1
D6: R=27.9507017179653; w_U=0.541728524905921; w_pi=0.301867578554025;
B=23.64225374467
D10: R=91.9452838579187; w_U=0.99715830614309; w_pi=0.993009065665522;
B=7018.05582412408
G1: R=6.17283949624454; w_U=0.201837236145844; w_pi=0.0666666665594411;
B=5.05754578605535
G4: R=63.6999391814617; w_U=0.826855819812353; w_pi=0.687959343159786;
B=95.5106684979231
G10: R=92.5923708630398; w_U=0.999998747977069; w_pi=0.999997605320831;
B=15974128.3188332
Checks in section: 78/78 passed

SUPPORTING EXAMPLES: enumeration or explicit formulas as indicated
A2 original R: 1.93548387096774
A2 swapped R: 3.52941176470588
A5 target B: 283.5
A5 other B: 0.160475482912333
A5 target event genotype alternative: 0.00296296296296
A5 B person-alternative mean: 5.31210320140484
A5 target R: 46.2
A5 other R: 0.162962962962963
B1 source posterior K=100: 0.99990100980003
B1 source posterior K=10000: 0.99009999019801
B1 source posterior K=100000: 0.500000250000125
B1 occupancy: 0.63212037488873
B2 population nuisance B: 10.4709141274238
B2 uniform nuisance B: 10.8
B3 0.0144 power 15: 2.3737631379977e-28
B3 0.0862 power 15: 1.07797574067054e-16
B3 0.1156 power 15: 8.79766683331783e-15
B4 child: 0.025984

```

```
B4 unrelated: 0.000675168256
Checks in section: 32/32 passed

TOTAL: 132/132 checks passed
Check count includes related assertions, not independent validations.
```

In the output:

- **R** is the common-model genotype-replacement LR defined in manuscript Section 2.1.
- **w_U** is the target-genotype posterior under a uniform target-genotype prior.
- **w_{pi}** is the target-genotype posterior under the alternative-person population prior.
- **B** is the uniform-reference different-genotype comparison defined in manuscript Section 4.3.
- **A2 original** and **A2 swapped** identify the two likelihood assignments in Appendix A.2.
- The labels **A1–A6** and **B1–B4** refer to manuscript appendix subsections, not figure numbers.

The matched identity is

$$R = \frac{w_{\pi}(t)}{f(t)}.$$

It decomposes the same LR under compatible states, likelihoods, and nuisance treatment. It is not an independent statistic correcting the LR. The uniform-prior weight $w_U(t)$ cannot be substituted for $w_{\pi}(t)$.

S5. Figure inventory

The four figures are embedded in the current [manuscript.md](#) and included in its corresponding PDF. The separate SVG files are provided through the [Figures repository](#).

Open that repository and obtain the files by the names below. The repository address is not a direct download link to any individual figure. The manuscript, supplement, guide, and separate SVG files need not share one hosting directory for readers to use these repository links.

Figure	File	Manuscript location	Content
1	figure-1.svg	Section 3.4	Separate compatible pairs and incompatibility of A and B as the complete pair
2	figure-2.svg	Section 5	Exact-target, strict-compatibility, and allele-set population filters
3	figure-3.svg	Section 6	Example D: fixed target rarity with changing minor signal
4	figure-4.svg	Section 7	Example G: the same allele channels with changing quantitative patterns

S5.1 Figure 1

The diagram displays one locus of the strict two-contributor construction. Allele-copy symbols are not peak heights.

A has genotype $2/3$ and requires partner $1/4$. B has genotype $1/3$ and requires partner $2/4$. The complete named pair lacks observed allele 4.

The displayed full-profile LR uses thirteen independent loci. The joint zero applies to the complete named pair under complete observation without dropout, drop-in, or error. It is not a general exclusion of either person under larger mixtures or other observation mechanisms.

S5.2 Figure 2

The nested sets represent different population events:

- Target genotype $2/3$: probability 0.0144.
- Six strict two-person-compatible heterozygotes: total probability 0.0862.
- Ten genotypes with both alleles in the observed set: total probability 0.1156.

Cell areas do not represent probabilities. These population probabilities are not posterior probabilities of personal contribution.

S5.3 Figures 3 and 4

The peak heights represent the deterministic channel signals in manuscript Section 14.3. Peak shapes and widths are schematic and do not enter the likelihood calculation.

The SVG peak scales can be related to channel signals as follows:

Figure	Vertical scale	Baseline coordinate
3: example D	350 SVG coordinate units per signal unit	$y = 410$
4: example G	550 SVG coordinate units per signal unit	$y = 410$

Thus, a positive channel signal s has a peak apex at $410 - 350s$ in Figure 3 and $410 - 550s$ in Figure 4, subject to decimal rounding in the SVG coordinates. Zero channels in Figure 3 are represented by short baseline marks.

The target genotype is $5/6$, with population probability 0.0108, throughout both figures. The displayed weight uses a uniform target-genotype prior and is not a named-person contribution probability. Other contributor genotypes in G are population-weighted.

The Gaussian observation model has independent channel errors with $\sigma = 0.035$. It contains no laboratory dropout, drop-in, stutter, degradation, or detection-threshold mechanism. The illustrated settings are not independent loci.

S5.4 Display and printing

The SVGs are vector images and can be enlarged without raster pixelation. Use a display or print size that keeps annotations legible; a landscape figure supplement is suitable for full-size presentation.

Markdown viewers and AI systems differ in their handling of embedded SVG images and mathematical notation. Use the corresponding PDF or open the separate SVG files in a compatible viewer if the embedded figures are not displayed.

PDF text extraction can disrupt equation and table reading order. For exact mathematical notation, consult the controlling Markdown or an inspectable PDF page rather than relying on uncertain extracted text.

The complete captions are in the manuscript. Figures should be read with those captions and the corresponding model specifications.